

# Learning Visual Feature-Based World Models via Residual Latent Action

Xinyu Zhang<sup>1</sup> Zhengtong Xu<sup>2</sup> Yutian Tao<sup>3</sup>  
 Yeping Wang<sup>3</sup> Yu She<sup>2</sup> Abdeslam Boularias<sup>1</sup>

<sup>1</sup>Rutgers University

<sup>2</sup>Purdue University

<sup>3</sup>University of Wisconsin-Madison

## Abstract

World models predict future transitions from observations and actions. Existing works predominantly focus on image generation only. Visual feature-based world models, on the other hand, predict future visual features instead of raw video pixels, offering a promising alternative that is more efficient and less prone to hallucination. However, current feature-based approaches rely on direct regression, which leads to blurry or collapsed predictions in complex interactions, while generative modeling in high-dimensional feature spaces still remains challenging. In this work, we discover that a new type of latent action representation, which we refer to as *Residual Latent Action* (RLA), can be easily learned from DINO residuals. We also show that RLA is predictive, generalizable, and encodes temporal progression. Building on RLA, we propose *RLA World Model* (RLA-WM), which predicts RLA values via flow matching. RLA-WM outperforms both state-of-the-art feature-based and video-diffusion world models on simulation and real-world datasets, while being orders of magnitude faster than video diffusion. Furthermore, we develop two robot learning techniques that use RLA-WM to improve policy learning. The first one is a minimalist world action model with RLA that learns from actionless demonstration videos. The second one is the first visual RL framework trained entirely inside a world model learned from offline videos only, using a video-aligned reward and no online interactions or handcrafted rewards.

## 1 Introduction

World models have recently received increasing research attention due to their great potential for policy learning and reasoning through future state prediction [1]. Currently, the predominant paradigm in world modeling relies on video generation, by predicting future trajectories in pixel-aligned VAE latent spaces [2–7]. While visually compelling, this approach is prone to hallucination [8] and suffers from a heavy computational overhead [9]. As a result, downstream applications of world models remain largely constrained to open-loop robot data generation [10, 11], policy pretraining [12, 13], and planning for specific tasks [14–16].

Visual feature-based world models predict features of future frames, such as DINO tokens, rather than just videos [17]. This direction is partly motivated by studies in cognitive science showing that humans do not reason in raw pixels but in latent spaces shaped by task goals and physical understanding [18, 19]. DINO-WM [16, 20, 21] shows that direct regression of future DINO tokens leads to efficient and accurate world models for 2D manipulation tasks. However, despite these advantages, feature-based world models remain far less adopted, as predictions often become blurry or even collapse in complex 3D interactions [10]. A seemingly straightforward solution is to use generative models in feature space. However, feature-space generation is even more difficult than in

Project page: <https://mlzxy.github.io/rla-wm>

pixel space due to the higher dimensionality [22, 23]. More importantly, heavy generative pipelines undermine the very advantages that feature-based models should provide, as detailed in Sec. 3.

Motivated by these challenges, we seek to answer two key questions: (1) how to develop an efficient yet accurate world model in a visual feature space that scales to complex 3D manipulation? and (2) how to leverage such world models to improve downstream policies?

While visual features are high-dimensional, we believe the manifold of valid physical transitions is inherently lower-dimensional. Therefore, learning a compact representation of these low-dimensional dynamics would enable a more principled approach to visual feature-based world models. In this work, we introduce *Residual Latent Actions* (RLA). RLA is deceptively simple: it encodes the residual between DINO tokens of two frames ( $s_t, s_{t+h}$ ) into a compact latent vector, and is trained with a single regression loss to reconstruct  $s_{t+h}$  from  $s_t$ , as shown in Fig. 1a. Despite its simplicity, we find that RLA exhibits three surprising empirical properties that make it well-suited for dynamics learning. (1) RLA is sufficiently predictive. As illustrated in Fig. A1, the decoder  $f_{\text{dec}}$  can accurately reconstruct  $s_{t+h}$  from RLA  $z$  and  $s_t$  in a single forward pass. In contrast, prior methods mainly use latent actions as weak conditioning labels for iterative generation [24, 25]. (2) RLA generalizes to novel scenes and motion patterns, even when trained on limited data, as shown in Fig. A3. (3) RLA latent space exhibits a temporal topology; although training is performed only on frame pairs, decoding linear interpolations between a Gaussian noise and RLA yields results that approximate intermediate frames, as illustrated in Fig. A2.

Based on RLA, we propose the *RLA World Model* (RLA-WM), shown in Fig. 2. Instead of directly regressing DINO tokens  $s_{t+h}$ , RLA-WM first predicts RLA  $z$  via flow matching with  $s_t$  and actions  $a_{t:t+h}$  as input conditions, then predicts  $s_{t+h}$  from  $s_t$  and  $z$ . RLA-WM significantly outperforms state-of-the-art feature-based and video diffusion world models on both simulation and real-world datasets, while remaining more efficient as the flow matching runs in the compact RLA space.

Furthermore, we introduce two robot learning techniques built on RLA and RLA-WM. First, we show that a behavior cloning policy can be extended into a minimalist world action model (WAM) using a single linear layer that predicts RLA from the current observation. Unlike prior WAMs that couple action prediction with heavy video generation backbones [7], our approach imposes no such coupling, adds no inference cost, and consistently improves policy success rates for imitation learning from actionless videos. Second, we present the first demonstration of visual reinforcement learning (RL) entirely inside a world model learned from a small offline video dataset without online interactions, handcrafted rewards, or even auxiliary BC loss during RL. Our World Model-based RL (WMRL) yields a significant improvement on ManiSkill tasks for the XArm and UR10e robots.

Our contributions are threefold. (1) We propose the Residual Latent Action (RLA), a simple latent action representation learned from DINO residuals. (2) We present RLA-WM, which predicts RLA via flow matching and sets a new state-of-the-art among visual feature-based world models. (3) We demonstrate the value of RLA and RLA-WM in two novel applications: (a) a minimalist world action model that learns from actionless videos; (b) a visual reinforcement learning framework that optimizes policy via rollouts in the RLA-WM.

## 2 Related Work

**World Models for Robotics.** Learning world models from offline datasets has emerged as a promising paradigm for future state prediction in robotics [26, 2]. Existing approaches largely focus on predicting future videos [3–7] and 3D geometry, such as point clouds [27–30]. Despite their success, video prediction induces a heavy computational overhead due to diffusion models. While 3D world models benefit from spatial priors, their structural assumptions often limit them to specific tasks. Another line of research explores learning world models via online rollouts within simulators [31–36], but the reliance on simulators and handcrafted reward functions limits their application.

**World Models in Visual Feature Space.** An alternative to pixel-space prediction is embedding future states in a learned visual feature space. For instance, V-JEPA predicts future features for self-supervised learning [17, 37]. The DINO-WM family of world models [16, 20, 21] predicts DINO tokens [38, 39] of future frames through a direct regression. DINO-WM [16] shows that predicting in a feature space mitigates the need for heavy generative models for 2D robot manipulation tasks. However, for complex 3D manipulation, we observe that simply applying regression in the feature

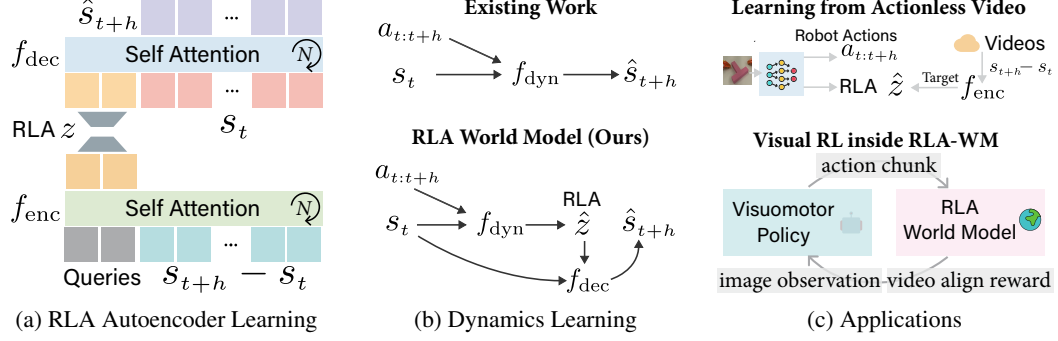


Figure 1: **Overview of our framework.** We introduce the Residual Latent Action (RLA), which compresses the DINO token residual  $s_{t+h} - s_t$  into a compact latent  $z$ . We discover that RLA is predictive, generalizable, and encodes temporal progression. Next, we propose the RLA World Model (RLA-WM), which learns from offline videos and predicts RLA  $z$  instead of  $s_{t+h}$  directly. RLA-WM achieves accurate future prediction while being more efficient than state-of-the-art feature-based and video diffusion world models. Our approach enables two applications: learning policies from actionless videos and visual reinforcement learning via interaction entirely within RLA-WM.

space often yields blurred or collapsed estimations. In contrast, our approach avoids regression-to-the-mean, enabling efficient and accurate multi-modal prediction of DINO tokens in future frames.

**Latent Actions.** Learning compact latent actions from videos has emerged as a popular technique in robot learning [40–44, 24, 25]. Existing approaches fall into two categories. The first leverages latent actions as proxy controls for imitation learning from actionless videos where proprioceptive data are absent [41–44]. The second utilizes latent actions as weak condition labels for video diffusion [24, 25]. In contrast, we learn Residual Latent Action (RLA) from DINO residuals instead of raw pixels. RLA outperforms existing methods [24, 44] as a better action proxy (Sec. 4.2), without requiring diffusion, and can be decoded into DINO tokens of future frames in a single feedforward pass.

### 3 Method

**Problem Formulation.** Let  $x_t \in \mathbb{R}^{H \times W \times 3}$  denote the raw image observation at time  $t$ . We represent the DINO patch tokens as  $s_t \in \mathbb{R}^{L \times C}$ , where  $C$  is the feature dimension and  $L = \frac{H \times W}{P^2}$  is the sequence length for a given patch size  $P$ . We define an action chunk of horizon  $h$  at time  $t$  as  $a_{t:t+h}$ . Our objective is to learn a dynamics function  $f_{\text{dyn}} : (s_t, a_{t:t+h}) \mapsto s_{t+h}$  using only raw offline videos, without online rollouts or access to handcrafted reward functions or labels. This function acts as a direct, multi-step world model in the feature space.

**Learning Latent Actions on DINO Residuals.** The physical world is inherently uncertain, which makes the dynamics function  $f_{\text{dyn}}$  highly multi-modal. That is, given an image  $x_t$  and actions  $a_{t:t+h}$ , there can be multiple valid values for  $x_{t+h}$ . Prior work addresses this through generative video models to predict  $\hat{x}_{t+h}$ . However, these methods are computationally heavy and prone to hallucination [8]. Pioneering works such as DINO-WM and JEPA instead learn world models in a feature space, such as DINO tokens, which is more efficient, does not require diffusion, and reduces hallucination because visual features encode rich semantic and geometric information [16]. These works motivate us to design a world model that, given  $s_t$ , directly predicts  $\hat{s}_{t+h}$  in DINO token space rather than predicting pixel-level  $\hat{x}_{t+h}$ . However, despite the impressive results of DINO-WM, the key limitation is its direct regression design, which is computationally efficient but often results in blurry or collapsed predictions in complex 3D interactions.

A straightforward solution is to revert to generative models, such as diffusion or flow matching, but in the feature space, to predict  $\hat{s}_{t+h}$ . However, a counter-intuitive yet critical fact is that DINO tokens (and ViT or ResNet features generally) have a far higher dimensionality than the pixel-aligned VAE latents used in image or video generation. For a  $512 \times 512$  image, Stable Diffusion VAE [45] yields roughly  $64^2 \times 4 \approx 16\text{k}$  dimensions, whereas DINOv3-L tokens produce  $32^2 \times 1024 \approx 1\text{M}$  dimensions, nearly two orders of magnitude larger. This curse of dimensionality makes generative modeling of DINO tokens highly challenging [22]. While RAE [23] proposes diffusion techniques

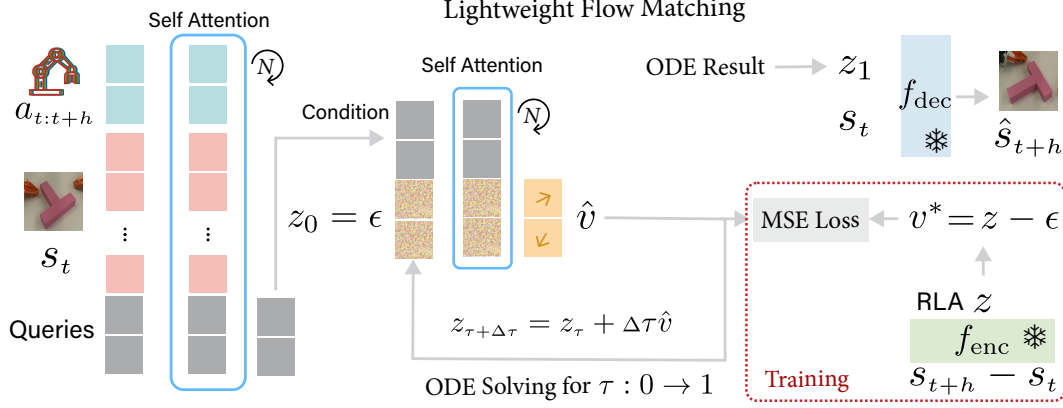


Figure 2: **RLA World Model.** RLA-WM predicts future states by generating the residual latent action  $z$ . We first embed the robot actions  $a_{t:t+h}$  (padded to a maximum horizon) via an MLP. This embedding is concatenated with the DINO tokens  $s_t$  and learnable queries, then processed through self-attention layers to produce condition tokens. During flow matching, this condition is fixed and concatenated with a noisy latent  $z_\tau$ , starting from Gaussian noise  $z_0 = \epsilon$ . The flow network predicts the velocity  $\hat{v}$  to iteratively transform  $z_0$  into the final RLA  $z_1$ . Finally,  $f_{\text{dec}}$  decodes  $\hat{s}_{t+h}$  from  $z_1$  and  $s_t$ . During training, the model is supervised by the MSE loss against the ground truth velocity  $v^* = z - \epsilon$ , requiring no feature or image reconstruction losses.

to generate DINO tokens from noise and class labels, it is not widely adopted because adapting it to a dynamics learning setting is not trivial, as shown in Tab. 1. More importantly, using heavy generative models defeats the purpose of feature-space learning, as they undermine both the computational efficiency and the reduced hallucination that feature-based world models offer.

To address this challenge, we shift focus from directly generating  $\hat{s}_{t+h}$  to learning a representation that captures the transition from  $s_t$  to  $s_{t+h}$ . We propose learning this representation from DINO token residuals  $s_{t+h} - s_t$ , which also corresponds to the flow matching velocity of a Schrödinger bridge [46, 47] from  $s_t$  to  $s_{t+h}$ . Specifically, we feed these residuals along with learnable queries into an encoder  $f_{\text{enc}}$ , project the output queries to a low-dimensional space to obtain  $z$ , and pass  $z$  along with  $s_t$  into a decoder  $f_{\text{dec}}$  to reconstruct  $s_{t+h}$  (Fig. 1a). We refer to  $z$  as a *Residual Latent Action* (RLA). The RLA autoencoder uses almost only self-attention and a single regression loss on  $s_{t+h}$ . There are three key properties of RLA that set it apart from prior work, making it an ideal representation for dynamics modeling:

**Predictive Sufficiency** Unlike in prior work, where latent actions serve only as weak conditioning for diffusion, RLA does not require iterative generation. We find that our RLA decoder  $f_{\text{dec}}$ , when conditioned on a compact RLA  $z$ , is able to reconstruct future DINO tokens with high fidelity in a single feedforward pass. Reconstruction examples are provided in Fig. A1.

**Generalizability** RLA autoencoder generalizes to novel scenes. In Sec. 4.2, we demonstrate this by training RLA on task-agnostic videos and applying it to task-relevant, actionless videos for imitation learning. Examples of encoding unseen robot object interactions are provided in Fig. A3.

**Temporal Topology** An emergent property of RLA is the topology of its learned latent space. Although the autoencoder is only trained on frame pairs  $(s_t, s_{t+h})$ , the RLA latent space naturally encodes temporal progression. Interpolating between a Gaussian noise and RLA produces frames that correspond to temporally intermediate states, as shown in Fig. A2.

**RLA World Model.** Based on RLA, we revisit feature-based world modeling. Learning neural dynamics in RLA space encourages the model to capture state evolution rather than absolute states. This aligns with classical physics simulation, which models relative mesh displacements [48]. Motivated by this, instead of generating high-dimensional  $s_{t+h}$ , we propose a world model to predict the compact RLA  $z$ , which is then decoded with current state  $s_t$  to reconstruct  $s_{t+h}$ . Specifically, learnable queries are concatenated with  $s_t$  and embedded actions  $a_{t:t+h}$  and transformed via self-attention. These query tokens are then concatenated with a noisy RLA  $z_\tau = \tau z + (1 - \tau)\epsilon$ , where  $\epsilon \sim \mathcal{N}(0, I)$ , through subsequent self-attention layers to predict the velocity  $\hat{v}$ . During training, we supervise with ground truth velocity  $v^* = z - \epsilon$ , where  $z = f_{\text{enc}}(s_{t+h} - s_t)$ . At inference, we



Table 1: **Evaluation of Future Frame Prediction.** We evaluate RLA-WM by predicting  $\hat{s}_{t+h}$  given  $s_t$  and actions  $a_{t:t+h}$ . We report LPIPS, SSIM, L1 distance to ground-truth DINO tokens  $s_{t+h}$  (DINO L1), and FLOPs per inference. Best results are **bold**, and second-best results are underlined.

| Model         | ManiSkill [49] |              |              | IWS [10]     |              |              | FLOPs ↓     |
|---------------|----------------|--------------|--------------|--------------|--------------|--------------|-------------|
|               | LPIPS ↓        | SSIM ↑       | DINO L1 ↓    | LPIPS ↓      | SSIM ↑       | DINO L1 ↓    |             |
| DINO-WM [16]  | 0.156          | 0.865        | 0.078        | <u>0.223</u> | <u>0.825</u> | <u>0.058</u> | <b>2.1T</b> |
| RAE [23]      | 0.324          | 0.717        | 0.143        | 0.550        | 0.625        | 0.159        | 14.3T       |
| FM-WM [50]    | <u>0.127</u>   | <u>0.890</u> | <u>0.063</u> | 0.360        | 0.741        | 0.119        | 14.3T       |
| Vid2World [6] | 0.199          | 0.705        | 0.084        | 0.388        | 0.710        | 0.139        | 1.1P        |
| RLA-WM (Ours) | <b>0.071</b>   | <b>0.931</b> | <b>0.030</b> | <b>0.196</b> | <b>0.847</b> | <b>0.053</b> | <u>3.5T</u> |

sample  $z_0 = \epsilon$  and solve the ODE with  $z_{\tau+\Delta\tau} = z_\tau + \Delta\tau\hat{v}$  from  $\tau = 0$  to 1. The final  $z_1$  is the predicted RLA, and is decoded via  $f_{\text{dec}}$  with  $s_t$ . Since the condition network is executed once and iterative generation remains within the compact RLA space, the flow matching is lightweight, as shown by the floating point operations (FLOPs) reported in Tab. 1. The compactness of the RLA space also helps the model to predict long-term dynamics more accurately, without over-attention to the excessive details in dense observation spaces. Our RLA-WM framework is illustrated in Fig. 2.

## 4 Experiments

Our experiments aim to answer two key questions: (1) Can RLA-WM perform accurate multi-step prediction in a visual feature space? (2) How can RLA and RLA-WM improve robot policies? To address the first question, we evaluate the RLA-WM on simulation and real-world robot manipulation videos, using image and feature prediction metrics across multi-step rollouts. For the second one, we provide two applications of RLA and RLA-WM: (a) extending behavior cloning (BC) policies to World Action Models (WAM) via RLA, and (b) performing visual RL entirely inside an RLA-WM.

### 4.1 Prediction Quality Evaluation

**Datasets.** The experiments are performed on the ManiSkill simulation suite [49] and the IWS real-world dataset [10]. In ManiSkill, we adopt three robot arms (Panda with parallel gripper, XArm with Robotiq gripper, UR10 with cylinder end-effector) across five built-in tasks: Pull Cube, Pull Cube with Tool, Roll Ball, Push T, and Poke Cube. We additionally curate a task-agnostic play environment where the robot freely interacts with primitive shapes without task-specific goals. Fig. A4 shows an overview of these environments. Unlike Dreamer [34], we do not use online interactions or rewards for training. We collect 1,000 successful and 500 failed episodes per ManiSkill task using pretrained state-based PPO agents, and 3,000 play videos per robot via scripted task and motion planning. For IWS, we select the three most challenging tasks (Push T, Rope Manipulation, and Open Box) using bimanual ALOHA robots, each providing over 600 human teleoperation demonstrations.

**Training and Evaluation.** RLA autoencoder is trained per dataset (ManiSkill and IWS), using a single model for multiple tasks and robots. Because each robot in ManiSkill and each scene in IWS has a different action space, as in Sec. A.2, we train the dynamics part of RLA-WM per robot on ManiSkill using task-relevant videos, and per task on IWS. For validation on ManiSkill, we use 10 success and 10 failure episodes, unseen during training, per task. For IWS, we use the official validation set. During training, we randomly sample a pair  $(s_t, s_{t+h})$  separated by a variable horizon  $h \in [1, 15]$ . The network predicts  $\hat{s}_{t+h}$  from  $s_t$  and actions  $a_{t:t+h}$ . All videos have a dimension of  $512 \times 512$ . During evaluation, we condition on an initial frame and autoregressively unroll predictions for 30 steps on ManiSkill (action chunk size 10) and 60 steps on IWS (chunk size 15), requiring 3 and 4 autoregressive steps, respectively. We measure the final frame’s fidelity against the ground truth using LPIPS [51], SSIM [52], and the L1 distance of DINO tokens.

**Baselines.** We benchmark RLA-WM against a suite of state-of-the-art visual feature-based and video diffusion-based world models: (1) **DINO-WM** [16], a regression network that predicts DINO tokens. We re-implement this method using DINOv3 features to regress  $s_{t+h}$  given  $s_t$  and an action chunk  $a_{t:t+h}$ ; (2) **Vid2World** [6], a high-fidelity video diffusion world model based on an action-conditioned DynamicCrafter [53] architecture with 1.1B trainable parameters; (3) **RAE** [23],

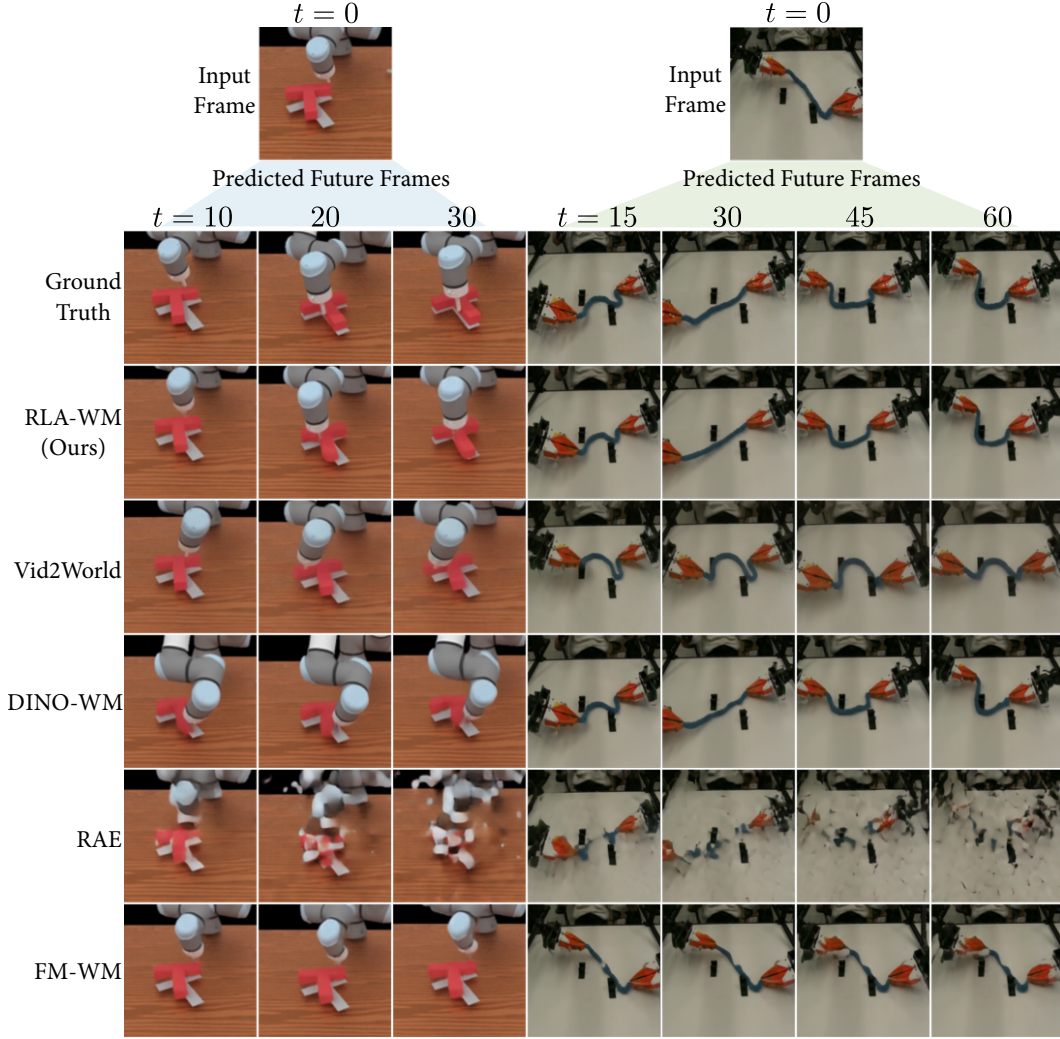


Figure 3: **Qualitative Comparison for RLA-WM.** Given an input frame at  $t = 0$ , our RLA-WM predicts future frames with high visual quality and physical fidelity, closely matching ground-truth states. In contrast, DINO-WM produces increasingly blurrier predictions for Push-T over longer horizons, and inconsistent rope states. Applying diffusion or flow matching directly in DINO token space yields inferior results (RAE, FM-WM). Vid2World generates visually sharp frames. However, it hallucinates and predicts physical states that diverge significantly from reality.

a diffusion-based model for DINO tokens which we adapt to incorporate  $s_t$  and  $a_{t:t+h}$  as conditional inputs within our transformer backbone; (4) **FM-WM**, a Flow Matching [50] baseline we implement to learn a conditional probability path that directly flows  $s_t$  to the future state  $s_{t+h}$  given  $a_{t:t+h}$ .

**Results.** As detailed in Tab. 1, RLA-WM significantly outperforms all feature-based (DINO-WM, RAE, FM-WM) and video diffusion (Vid2World) baselines across all measured metrics on both ManiSkill and IWS. We also provide qualitative comparisons on validation episodes (unseen during training) in Fig. 3 and Fig. A6 to A10. While Vid2World generates frames with sharp geometric and textural details, it hallucinates and predicts trajectories that lack physical grounding and diverge from reality, resulting in inferior metrics. Furthermore, Vid2World requires 1.1P FLOPs, a computational footprint nearly three orders of magnitude larger than our 3.5T FLOPs. RLA-WM achieves high-fidelity predictions with minimal hallucination and a computational efficiency second only to the direct regression of DINO-WM. This higher performance is enabled by RLA’s ability to perform flow matching within a compact latent space. Note that to compute image-space metrics, the DINO tokens  $\hat{s}_{t+h}$  are decoded to RGB via a pre-trained UNet [54].

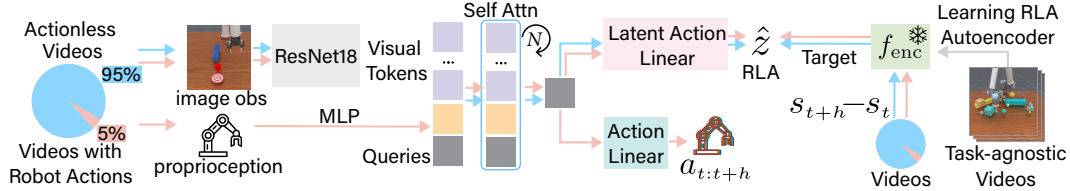


Figure 4: **Learning from Actionless Videos using RLA.** We extend a BC ResNet with a linear layer to predict the RLA  $\hat{z}$ . The RLA targets are extracted from  $(s_t, s_{t+h})$  using an RLA encoder  $f_{enc}$  learned from task-agnostic videos. This turns the BC policy into a minimalist world action model that can learn from videos whose proprioceptive states and robot actions are not available, without forcing the policy to couple with DINO or video generation backbones. As shown in Tab. 2, RLA outperforms other latent action learners (replacing  $f_{enc}$ ) within the same framework.

Table 2: **Latent Action Evaluation.** We report success rates (%) for imitation policies trained on actionless videos using latent actions. RLA achieves the highest average success rate and rank. Success rates are evaluated over 50 episodes (seeds 42-91) and averaged over the last five checkpoints.

| Method        | PushT       | Roll        | Pull        | Pull Tool   | Poke        | Rank $\downarrow$ | Avg SR $\uparrow$ |
|---------------|-------------|-------------|-------------|-------------|-------------|-------------------|-------------------|
| BC-ResNet     | 3.6         | 42.0        | 33.6        | 7.6         | 49.2        | 3.8               | 27.2              |
| DINO CLS [39] | 7.6         | 39.6        | 40.4        | 4.4         | 44.8        | 4.0               | 27.4              |
| UniVLA [44]   | 6.0         | 37.6        | 42.8        | 7.2         | 50.0        | 3.8               | 28.7              |
| AdaWorld [24] | 9.2         | 38.4        | <b>48.4</b> | 10.8        | 61.6        | 2.2               | 33.7              |
| RLA (Ours)    | <b>15.2</b> | <b>43.8</b> | 43.6        | <b>12.0</b> | <b>63.6</b> | <b>1.2</b>        | <b>35.6</b>       |

## 4.2 Minimalist World Action Model with RLA

**Architecture and Motivation.** World Action Models (WAMs) combine robot action prediction with future video generation in a hybrid architecture [7, 55]. WAMs can be used as robot policies and show improved performance compared to policies trained with action prediction alone. However, due to the complexity of video generation, existing architectures are often tightly coupled to heavy video backbones, while predicting actions via an auxiliary module. This coupling limits their flexibility. The proposed RLA model provides a flexible alternative. We propose a minimalist WAM by extending a standard ResNet-18 [56] behavior cloning (BC) policy. We first pre-train the RLA autoencoder entirely on task-agnostic play data. The BC network then takes a  $128 \times 128$  image observation and proprioceptive joint angles as input, projecting them into a shared feature. Next, the network branches into two linear heads, one predicts robot actions, and the other predicts the RLA  $z$ , which is supervised by the pre-trained RLA autoencoder. Our architecture is visualized in Fig. 4.

**Learning from Actionless Video.** We evaluate our minimalist WAM in a setting where only a small fraction of demonstrations contain action labels – a practical setup for scaling robot learning to large-scale, unlabeled videos. We only include robot actions and proprioceptive states for 5% of all videos (15% for PushT due to its difficulty). The remainder are actionless, video-only trajectories. During training, we construct each batch by sampling equally from videos with and without actions. For actionless videos, we mask the action loss, replace the proprioceptive input with a learnable default token, and train the shared backbone using the RLA  $z$  encoded from  $(s_t, s_{t+h})$ . During evaluation, we discard the RLA head and evaluate the policy’s success rate using only the action head.

**Baselines.** We benchmark RLA against several latent action formulations: (1) **DINO CLS**, which uses the DINO class token of the future frame  $s_{t+h}$  as the latent action. This follows Flare [41], which aligns visual features  $s_t$  to  $s_{t+h}$ . Note that Flare’s implementation is not publicly accessible. (2) **UniVLA** [44], which learns latent actions from DINO tokens of frame pairs  $(s_t, s_{t+h})$  using spatial-temporal attention and VQ-VAE [57]. (3) **AdaWorld** [24], which derives latent actions from frame pairs  $(x_t, x_{t+h})$  via a VAE and operates on raw RGB images with cross-attention. For evaluation, we report success rates over 50 evaluation episodes with standard seeds from 42 to 91. Each baseline’s latent extractor is pre-trained on the same dataset and substituted as the  $f_{enc}$  within our WAM framework in Fig. 4.

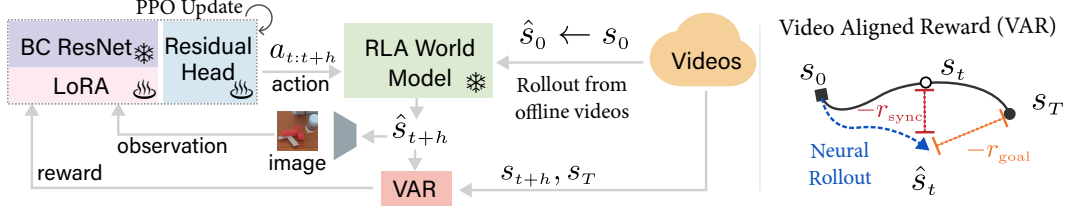


Figure 5: **Visual Reinforcement Learning within RLA World Models.** We adapt a pretrained ResNet BC policy for RL using LoRA adapters and a residual action head predicting delta actions and a Gaussian log deviation. The policy outputs action chunks  $a_{t:t+h}$  to our RLA-WM, which predicts future tokens  $\hat{s}_{t+h}$ . A pretrained UNet decodes  $\hat{s}_{t+h}$  into RGB observations for next step. RLA-WM resets its state  $\hat{s}_0$  from the initial frame  $s_0$  of a randomly sampled offline demonstration. For reward, we propose Video Aligned Reward (VAR). Because neural rollouts are time-synchronized with the reference video, VAR is simply defined as the negative DINO L1 distance between  $\hat{s}_t$  and  $s_t$  (or terminal  $s_T$ ). Policy is optimized via PPO using rollouts generated entirely inside RLA-WM.

**Results.** Table 2 presents quantitative comparisons. Our RLA as a latent action significantly outperforms existing methods, achieving +8.5% over the BC baseline and +1.9% over AdaWorld (second best), yielding the highest success rate on nearly all tasks. Notably, on the most challenging Push-T task, which requires spatial reasoning and long horizons, our method shows the largest improvement (15.2% over the baseline’s 3.6%), while AdaWorld (second best) achieves only 9.2%. The unique aspect of RLA is that adding one linear layer to predict  $z$  makes the simple framework (Fig. 4) a true world action model, as RLA enables accurate prediction of  $s_{t+h}$  from  $s_t$ .

### 4.3 Visual Reinforcement Learning within RLA World Model

**Motivation.** Extracting a robust visuomotor policy from a world model trained on offline videos remains a fundamental challenge [58]. Existing methods generally fall into two paradigms: reinforcement learning (RL) and planning. In RL, GWM [13] improves sample efficiency but still requires simulator interaction. UniSim [2] uses a video diffusion model as a simulator, but requires massive compute, remains unreleased, and is evaluated on a single task. Conversely, planning with world models lacks a unified framework: gradient-based methods suffer from non-convex optimization landscapes [16], and heuristic planners [59, 15] are highly task-specific. While principled methods like TD-MPC [33, 60] and the Dreamer [34] exist, they rely on pre-defined reward functions and large-scale online interactions. Thus, performing RL entirely within a learned world model is a critical open problem. This requires a world model that (1) accurately predicts the future, (2) supports efficient rollouts, and (3) operates without manual reward engineering.

**Architecture.** We introduce World Model-based RL (WMRL), a framework for visual RL fully within our RLA-WM. The RL policy is initialized from a pre-trained BC-ResNet policy, with LoRA adapters and a residual head added to predict Gaussian action distributions. The policy maps an RGB image at time  $t$  to an action chunk  $a_{t:t+h}$ , which the RLA-WM uses to predict  $\hat{s}_{t+h}$ . Previous works often employ SAC [61, 62] or spline [63] to ensure a strict multi-step RL formulation. We simply adopt PPO [64], treat the  $(s_t, a_{t:t+h}, s_{t+h})$  tuple as a single transition, and bypass intermediate advantage estimation. We propose to rollout from offline videos by setting  $\hat{s}_0$  to the first frame  $s_0$  of a sampled demonstration. For reward, we introduce Video Aligned Reward. Since each rollout is time-aligned with a reference offline video, we compute the reward as the negative L1 distance between the DINO tokens  $\hat{s}_t$  and the ground-truth  $s_t$  (or the terminal  $s_T$ ). We observe a neural-to-sim gap between images decoded by UNet and the simulator’s ray tracing. We reduce this gap by applying the UNet decoding as a preprocessing step. Our WMRL framework is outlined in Fig. 5.

**Evaluation and Results.** WMRL is an offline RL method [65] since all of the robot’s actions take place inside the learned world model, without any additional interactions with the real world, which are the key cost in RL. Therefore, independent training runs can be repeatedly performed and evaluated inside the world model for free, and the best-performing policy is retained. This allows us to determine whether WMRL can surpass the performance of imitation learning, without using any additional interaction data from the real environment. Specifically, we first train the BC-ResNet policy for 40 epochs, saving checkpoints at each epoch. We use a single seed (42), as BC is generally

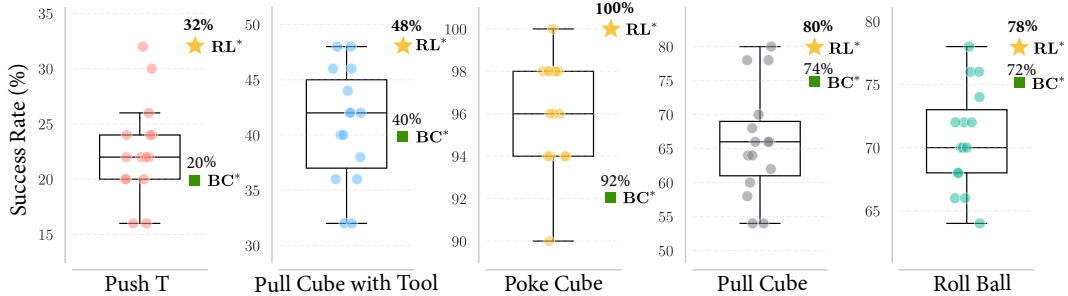


Figure 6: **World Model RL Performance Distribution.** We select the best-performing BC models (■ BC\*) and apply WMRL (Fig. 5) across 15 independent seeds (1-15). Each seed’s best checkpoint success rate is plotted as a dot (•), with the overall optimal checkpoint marked as ★ RL\*. Success rates here are evaluated on 50 episodes with standard seeds 42-91.

Table 3: **World Model RL Large-Scale Evaluation.** We extensively evaluate WMRL across 1500 episodes (seeds 1-1500) and report the success rates (%) of the best-performing models. With statistical significance, our WMRL achieves performance gain on both XArm and UR10e robots.

| Method          | XArm        |             | UR10e       |             | Panda          |  | Avg         |
|-----------------|-------------|-------------|-------------|-------------|----------------|--|-------------|
|                 | Poke Cube   | Roll Ball   | PushT       | Pull Cube   | Pull Cube Tool |  |             |
| BC-ResNet       | 89.9        | 65.5        | 17.2        | <b>84.5</b> | <b>41.1</b>    |  | 59.6        |
| RLA-WMRL (Ours) | <b>95.9</b> | <b>73.1</b> | <b>20.7</b> | 74.1        | 39.9           |  | <b>60.7</b> |

robust to initialization and batch shuffling [66]. Then, we apply WMRL on the BC policies. To account for variance, we run 15 trials (seeds 1-15) for 2,400 steps each, saving a checkpoint every 200 updates. Fig. 6 shows the performance distributions over 50 evaluation episodes with standard seeds (42-91). Our WMRL consistently improves the best-performing models over BC across all tasks. Further, in addition to the standard evaluation, we conduct a large-scale evaluation across 1500 seeds (1-1500). This directly addresses a commonly known limitation in many RL works [67, 68], where evaluations over a limited set of seeds can be unstable and prone to spurious results. As shown in Tab. 3, WMRL significantly improves policy performance on the XArm and UR10e robots. Although WMRL shows lower performance on the Panda robot, it achieves a statistically significant average gain of +1.1% over BC across the five tasks, using no additional data and only additional computation. We discuss the Panda performance in Sec. A.3. Nevertheless, one can always devise a simple meta algorithm that selectively uses WMRL depending on the task and setup, to improve BC policies without requiring any additional data, and only returns the highest-performing policies. We view our work as a preliminary but solid step toward rigorous standards for world model-based RL.

## 5 Limitations and Conclusion

We introduce Residual Latent Action, a compact representation of visual state dynamics, and propose RLA-WM, a state-of-the-art visual feature-based world model. Our framework enables two robot learning techniques: a minimalist world action model and a visual RL framework in RLA-WM. We believe our work is a strong step forward for visual feature-based world models. However, some limitations are still worth discussing: (1) Task-irrelevant background motion can cause visual changes between  $s_t$  and  $s_{t+h}$ , which are common for humanoids or eye-in-hand cameras. Forcing RLA to encode such information could waste representation capacity and degrade the quality of RLA. A solution is to learn view-independent RLA in the 3D space; (2) The visual changes between  $s_{t+h}$  and  $s_t$  can result from occlusion or partial observation, such as when the viewpoint changes, or an object disappears and later reappears. Understanding these motions requires reasoning over historical observations, which is difficult to capture from only a single frame pair  $(s_t, s_{t+h})$ . A promising solution is to extend RLA to multiple frames; (3) Our evaluation focuses on small-scale datasets and simulation. In doing so, we ensure reproducibility and isolate our method-driven gains from mere data scaling. While this demonstrates the efficacy of our method, adapting our framework to internet-scale data in an open-world setting remains an open question and promising direction.



## References

- [1] Jiahua Dong, Qi Lyu, Baichen Liu, Xudong Wang, Wenqi Liang, Duzhen Zhang, Jiahang Tu, Hongliu Li, Hanbin Zhao, Henghui Ding, et al. Learning to model the world: A survey of world models in artificial intelligence. 2026.
- [2] Mengjiao Yang, Yilun Du, Kamyar Ghasemipour, Jonathan Tompson, Dale Schuurmans, and Pieter Abbeel. Learning interactive real-world simulators. In *International Conference on Learning Representations (ICLR)*, 2024.
- [3] Jialong Wu, Shaofeng Yin, Ningya Feng, and Mingsheng Long. Rlvr-world: Training world models with reinforcement learning. *arXiv preprint arXiv:2505.13934*, 2025.
- [4] Haoyu Wu, Diankun Wu, Tianyu He, Junliang Guo, Yang Ye, Yueqi Duan, and Jiang Bian. Geometry forcing: Marrying video diffusion and 3d representation for consistent world modeling. *arXiv preprint arXiv:2507.07982*, 2025.
- [5] Yuang Wang, Chao Wen, Haoyu Guo, Sida Peng, Minghan Qin, Hujun Bao, Xiaowei Zhou, and Ruizhen Hu. Precise action-to-video generation through visual action prompts. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12713–12724, 2025.
- [6] Siqiao Huang, Jialong Wu, Qixing Zhou, Shangchen Miao, and Mingsheng Long. Vid2world: Crafting video diffusion models to interactive world models. *arXiv preprint arXiv:2505.14357*, 2025.
- [7] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, et al. Worldvla: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.
- [8] Hui Han, Siyuan Li, Jiaqi Chen, Yiwen Yuan, Yuling Wu, Yufan Deng, Chak Tou Leong, Hanwen Du, Junchen Fu, Youhua Li, et al. Video-bench: Human-aligned video generation benchmark. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 18858–18868, 2025.
- [9] Zangwei Zheng, Xiangyu Peng, Yuxuan Lou, Chenhui Shen, Tom Young, Xinying Guo, Binluo Wang, Hang Xu, Hongxin Liu, Mingyan Jiang, et al. Open-sora 2.0: Training a commercial-level video generation model in \$200 k. *arXiv preprint arXiv:2503.09642*, 2025.
- [10] Yixuan Wang, Rhythm Syed, Fangyu Wu, Mengchao Zhang, Aykut Onol, Jose Barreiros, Hooshang Nayyeri, Tony Dear, Huan Zhang, and Yunzhu Li. Interactive world simulator for robot policy training and evaluation. *arXiv preprint arXiv:2603.08546*, 2026.
- [11] GigaBrain Team, Angen Ye, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, Haoyun Li, Jie Li, Jiagang Zhu, Lv Feng, et al. Gigabrain-0: A world model-powered vision-language-action model. *arXiv preprint arXiv:2510.19430*, 2025.
- [12] Russell Mendonca, Shikhar Bahl, and Deepak Pathak. Structured world models from human videos. In *Robotics: Science and Systems (RSS)*, 2023.
- [13] Guanxing Lu, Baoxiong Jia, Puhao Li, Yixin Chen, Ziwei Wang, Yansong Tang, and Siyuan Huang. Gwm: Towards scalable gaussian world models for robotic manipulation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9263–9274, 2025.
- [14] Yilun Du, Mengjiao Yang, Pete Florence, Fei Xia, Ayzaan Wahid, Brian Ichter, Pierre Sermanet, Tianhe Yu, Pieter Abbeel, Joshua B Tenenbaum, et al. Video language planning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [15] DeLong Chen, Theo Moutakanni, Willy Chung, Yejin Bang, Ziwei Ji, Allen Bolourchi, and Pascale Fung. Planning with reasoning using vision language world model. *arXiv preprint arXiv:2509.02722*, 2025.
- [16] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning (ICML)*, 2025.

- [17] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [18] Daniel LK Yamins and James J DiCarlo. Using goal-driven deep learning models to understand sensory cortex. *Nature neuroscience*, 19(3):356–365, 2016.
- [19] Peter W Battaglia, Jessica B Hamrick, and Joshua B Tenenbaum. Simulation as an engine of physical scene understanding. *Proceedings of the national academy of sciences*, 110(45):18327–18332, 2013.
- [20] Junha Chun, Youngjoon Jeong, and Taesup Kim. Sparse imagination for efficient visual world model planning. *arXiv preprint arXiv:2506.01392*, 2025.
- [21] Federico Baldassarre, Marc Szafraniec, Basile Terver, Vasil Khalidov, Francisco Massa, Yann LeCun, Patrick Labatut, Maximilian Seitzer, and Piotr Bojanowski. Back to the features: Dino as a foundation for video world models. *arXiv preprint arXiv:2507.19468*, 2025.
- [22] Zhenxin Zheng and Zhenjie Zheng. Rethinking diffusion model in high dimension. *arXiv preprint arXiv:2503.08643*, 2025.
- [23] Boyang Zheng, Nanye Ma, Shengbang Tong, and Saining Xie. Diffusion transformers with representation autoencoders. *arXiv preprint arXiv:2510.11690*, 2025.
- [24] Shenyuan Gao, Siyuan Zhou, Yilun Du, Jun Zhang, and Chuang Gan. Adaworld: Learning adaptable world models with latent actions. In *International Conference on Machine Learning (ICML)*, 2025.
- [25] Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, Haitian Liu, Ruowen Zhao, Yao Feng, Chendong Xiang, Yinze Rong, et al. Motus: A unified latent action world model. *arXiv preprint arXiv:2512.13030*, 2025.
- [26] Haoyu Zhen, Xiaowen Qiu, Peihao Chen, Jincheng Yang, Xin Yan, Yilun Du, Yining Hong, and Chuang Gan. 3d-vla: A 3d vision-language-action generative world model. In *International Conference on Machine Learning (ICML)*, 2024.
- [27] Tengbo Yu, Guanxing Lu, Zaijia Yang, Haoyuan Deng, Season Si Chen, Jiwen Lu, Wenbo Ding, Guoqiang Hu, Yansong Tang, and Ziwei Wang. Manigaussian++: General robotic bi-manual manipulation with hierarchical gaussian world model. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12232–12239. IEEE, 2025.
- [28] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. Pointworld: Scaling 3d world models for in-the-wild robotic manipulation. *arXiv preprint arXiv:2601.03782*, 2026.
- [29] Suning Huang, Qianzhong Chen, Xiaohan Zhang, Jiankai Sun, and Mac Schwager. Particle-former: A 3d point cloud world model for multi-object, multi-material robotic manipulation. *arXiv preprint arXiv:2506.23126*, 2025.
- [30] Ying Chai, Litao Deng, Ruizhi Shao, Jiajun Zhang, Kangchen Lv, Liangjun Xing, Xiang Li, Hongwen Zhang, and Yebin Liu. Gaf: Gaussian action field as a 4d representation for dynamic world modeling in robotic manipulation. *arXiv preprint arXiv:2506.14135*, 2025.
- [31] Chenhao Li, Andreas Krause, and Marco Hutter. Robotic world model: A neural network simulator for robust policy optimization in robotics. *arXiv preprint arXiv:2501.10100*, 2025.
- [32] SV Jyothir, Siddhartha Jalagam, Yann LeCun, and Vlad Sobal. Gradient-based planning with world models. *arXiv preprint arXiv:2312.17227*, pages 703–708, 2023.
- [33] Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.
- [34] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.



- [35] Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models. *arXiv preprint arXiv:2509.24527*, 2025.
- [36] Ignat Georgiev, Varun Giridhar, Nicklas Hansen, and Animesh Garg. Pwm: Policy learning with multi-task world models. In *International Conference on Learning Representations (ICLR)*, 2025.
- [37] Lorenzo Mur-Labadia, Matthew Muckley, Amir Bar, Mido Assran, Koustuv Sinha, Mike Rabbat, Yann LeCun, Nicolas Ballas, and Adrien Bardes. V-jepa 2.1: Unlocking dense features in video self-supervised learning. *arXiv preprint arXiv:2603.14482*, 2026.
- [38] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- [39] Oriane Siméoni, Huy V Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025.
- [40] Chuheng Zhang, Tim Pearce, Pushi Zhang, Kaixin Wang, Xiaoyu Chen, Wei Shen, Li Zhao, and Jiang Bian. What do latent action models actually learn? *arXiv preprint arXiv:2506.15691*, 2025.
- [41] Ruijie Zheng, Jing Wang, Scott Reed, Johan Bjorck, Yu Fang, Fengyuan Hu, Joel Jang, Kaushil Kundalia, Zongyu Lin, Loic Magne, et al. Flare: Robot learning with implicit world modeling. *arXiv preprint arXiv:2505.15659*, 2025.
- [42] Seonghyeon Ye, Joel Jang, Byeongguk Jeon, Sejeun Joo, Jianwei Yang, Baolin Peng, Ajay Mandlekar, Reuben Tan, Yu-Wei Chao, Bill Yuchen Lin, et al. Latent action pretraining from videos. In *International Conference on Learning Representations (ICLR)*, 2025.
- [43] Bahey Tharwat, Yara Nasser, Ali Abouzeid, and Ian Reid. Latent action pretraining through world modeling. *arXiv preprint arXiv:2509.18428*, 2025.
- [44] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Univla: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025.
- [45] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first international conference on machine learning*, 2024.
- [46] Yuyang Shi, Valentin De Bortoli, Andrew Campbell, and Arnaud Doucet. Diffusion schrödinger bridge matching. *Advances in neural information processing systems*, 36:62183–62223, 2023.
- [47] Valentin De Bortoli, Iryna Korshunova, Andriy Mnih, and Arnaud Doucet. Schrodinger bridge flow for unpaired data translation. *Advances in Neural Information Processing Systems*, 37:103384–103441, 2024.
- [48] Peter Yichen Chen, Jinxu Xiang, Dong Heon Cho, Yue Chang, GA Pershing, Henrique Teles Maia, Maurizio M Chiaramonte, Kevin Carlberg, and Eitan Grinspun. Crom: Continuous reduced-order modeling of pdes using implicit neural representations. *arXiv preprint arXiv:2206.02607*, 2022.
- [49] Tongzhou Mu, Zhan Ling, Fanbo Xiang, Derek Cathera Yang, Xuanlin Li, Stone Tao, Zhiao Huang, Zhiwei Jia, and Hao Su. Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.
- [50] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*.

- [51] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018.
- [52] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004.
- [53] Jinbo Xing, Menghan Xia, Yong Zhang, Haoxin Chen, Wangbo Yu, Hanyuan Liu, Gongye Liu, Xintao Wang, Ying Shan, and Tien-Tsin Wong. Dynamicrafter: Animating open-domain images with video diffusion priors. In *European Conference on Computer Vision*, pages 399–417. Springer, 2024.
- [54] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.
- [55] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-wam: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.
- [56] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [57] Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.
- [58] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3):440, 2018.
- [59] Jacob Berg, Chuning Zhu, Yanda Bao, Ishan Durugkar, and Abhishek Gupta. Semantic world models. *arXiv preprint arXiv:2510.19818*, 2025.
- [60] Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. *arXiv preprint arXiv:2203.04955*, 2022.
- [61] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. PMLR, 2018.
- [62] Qiyang Li, Zhiyuan Zhou, and Sergey Levine. Reinforcement learning with action chunking. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [63] Ge Li, Dong Tian, Hongyi Zhou, Xinkai Jiang, Rudolf Lioutikov, and Gerhard Neumann. Top-erl: Transformer-based off-policy episodic reinforcement learning.
- [64] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [65] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.
- [66] Dylan J Foster, Adam Block, and Dipendra Misra. Is behavior cloning all you need? understanding horizon in imitation learning. *Advances in Neural Information Processing Systems*, 37:120602–120666, 2024.
- [67] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [68] Horia Mania, Aurelia Guy, and Benjamin Recht. Simple random search provides a competitive approach to reinforcement learning. *arXiv preprint arXiv:1803.07055*, 2018.
- [69] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*.

- [70] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. Structured 3d latents for scalable and versatile 3d generation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 21469–21480, 2025.
- [71] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [72] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

## A Appendix

### A.1 Code

The source code of our work is included in the supplementary folder Code. Please see Code/README.md for instructions on installation, dataset setup, downloading pre-trained models, running the demo Jupyter notebook, and training.

### A.2 Implementation Details

Here we summarize the details and hyperparameters for each section of our experiments. Note that we use DINOv3-Large with channel size 1024, and AdamW optimizer [69] for all experiments.

Our RLA autoencoder training code is built on TRELLIS [70], which is released under the MIT License. Our imitation learning code is built on Diffusion Policy [71], also released under the MIT License. We use pre-trained DINOv3 models [39], which are released under the DINOv3 License. We use the ManiSkill rigid-body simulation suite [49] (Apache 2.0 License; assets under CC BY-NC 4.0) and the IWS dataset [10] (MIT License).

**RLA Autoencoder.** We use 12 self-attention layers for both  $f_{\text{enc}}$  and  $f_{\text{dec}}$ , with 16 heads and a channel size of 1024. Input images are  $512 \times 512$ . During training, we randomly sample frame pairs within a horizon of 200 for the IWS dataset and 100 for ManiSkill. We use a batch size of 128, a learning rate of  $10^{-4}$ , and train for 100k steps. Reconstruction of  $s_{t+h}$  uses both L1 and MSE loss, each with weight 1.0. On ManiSkill, we bias frame sampling toward object movement: with probability 0.9 we sample a frame pair containing object movement (object movements are pre-recorded in the simulator), otherwise we sample a random frame. On IWS (a real-world dataset), we sample frames uniformly at random. Unless specified otherwise, all experiments use an RLA size  $|z| = 2048$ . Specifically, the encoder output uses 32 query tokens, each projected to dimension 64. To render DINO tokens back to images, we train a separate decoder-only UNet with 4 upsampling deconvolution blocks; each block doubles the spatial resolution and halves the channel dimension.

**RLA World Model.** The condition network uses 8 self-attention layers with 16 heads, channel size 1024, and 32 query tokens. The flow matching network also uses 8 self-attention layers and channel size 1024, but operates on a smaller token count: only 64 tokens total (32 from the condition network and 32 for the noisy RLA  $z_\tau$ ), and its output dimension is 64, consistent with our RLA representation where  $|z| = 2048 = 32 \times 64$ . We use a maximum action horizon of 15. The RLA-WM is trained for 100k steps with learning rate  $10^{-4}$  and batch size 64. During inference, we use 30 Euler ODE steps for flow matching. Different robots have different action sizes due to their kinematics: for ManiSkill, Panda (8), UR10 (5), XArm-Robotiq (12); for the IWS ALOHA robot, we use the dataset’s provided actions — the rope task has action size 14 (7 joints for each ALOHA arm), the box task has action size 8 (3D Cartesian coordinates of each gripper, plus an additional dimension to control gripper openness), the Push-T task has action size 4 (2D table coordinates for each of two arms). Before feeding actions into the condition network, we embed them via a robot-specific MLP: actions are first padded to the maximum horizon (e.g., for Panda with horizon 15, we pad to shape  $8 \times 15$ ), then passed through the MLP. RLA-WM and its autoencoder each take 3 days on  $4 \times$  A6000 GPUs (48GB) with 256GB RAM. The dynamics component of RLA-WM is trained per robot on ManiSkill, per scene on IWS, while the RLA autoencoder is trained per dataset.

**Learning from Actionless Videos with RLA.** Input images are  $128 \times 128$ , and we use an action chunk size of 12 during training and pick the first 8 to execute each step during inference. The visual token self-attention uses 6 layers with channel size 256 and 8 heads. Training runs for 40 epochs with batch size 64, learning rate  $3 \times 10^{-4}$ , weight decay  $10^{-4}$ , and a cosine learning rate scheduler. For the BC-ResNet baseline, which uses only 5% of videos (those with actions), the epoch size is much smaller; we therefore scale up the number of epochs to match the total iterations of latent action-based training while keeping the same evaluation and checkpoint frequency (40 times). We save a checkpoint and evaluate after each epoch. Evaluation runs 50 episodes (random seeds 42-91), each with a maximum of 100 steps. Each policy is trained per task. A single training trial takes one day on an A4500 Ada GPU (24GB).

**Visual RL within RLA-WM.** We inject LoRA adapters into all linear and convolutional layers of the pre-trained BC ResNet policy. Unlike the architecture used in our learning from actionless video experiments, this policy does not use self-attention over spatial tokens; instead, we apply global

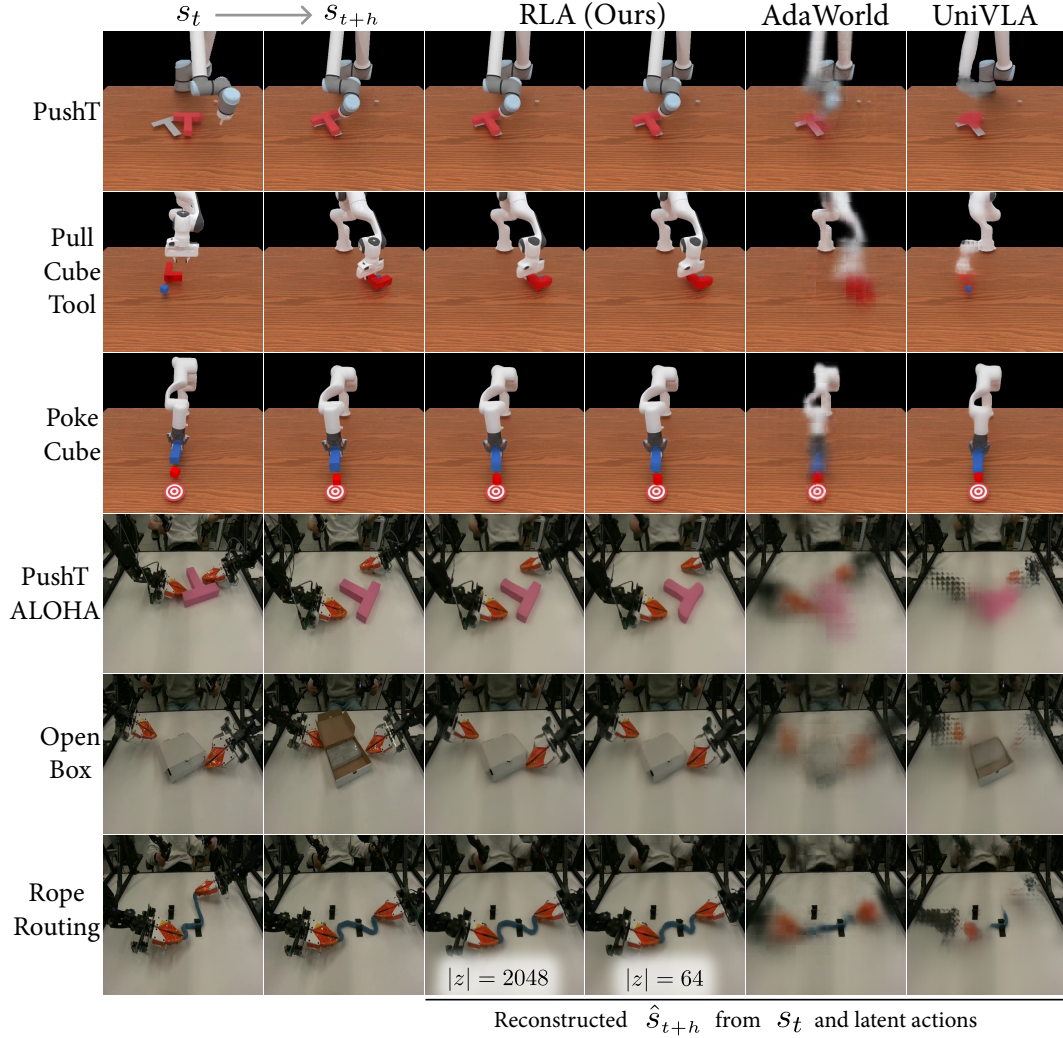


Figure A1: **Reconstruction Quality Comparison.** We compare the reconstruction  $\hat{s}_{t+h}$  decoded by  $f_{\text{dec}}$  from the current frame  $s_t$  and a latent action  $z$ , where  $z$  is encoded by  $f_{\text{enc}}$  from the pair  $(s_t, s_{t+h})$ . RLA produces accurate reconstructions even with a compact latent dimension  $|z| = 64$ . This is particularly impressive given the  $1024 \times 1024$  dimension of DINO token  $s_{t+h}$ . In contrast, AdaWorld [24] and UniVLA [44] yield severely blurred and inaccurate predictions with latent dimensions 2048 and 256, respectively. This demonstrates that RLA captures sufficient predictive information for dynamics and enables accurate future token decoding in a single feedforward pass.

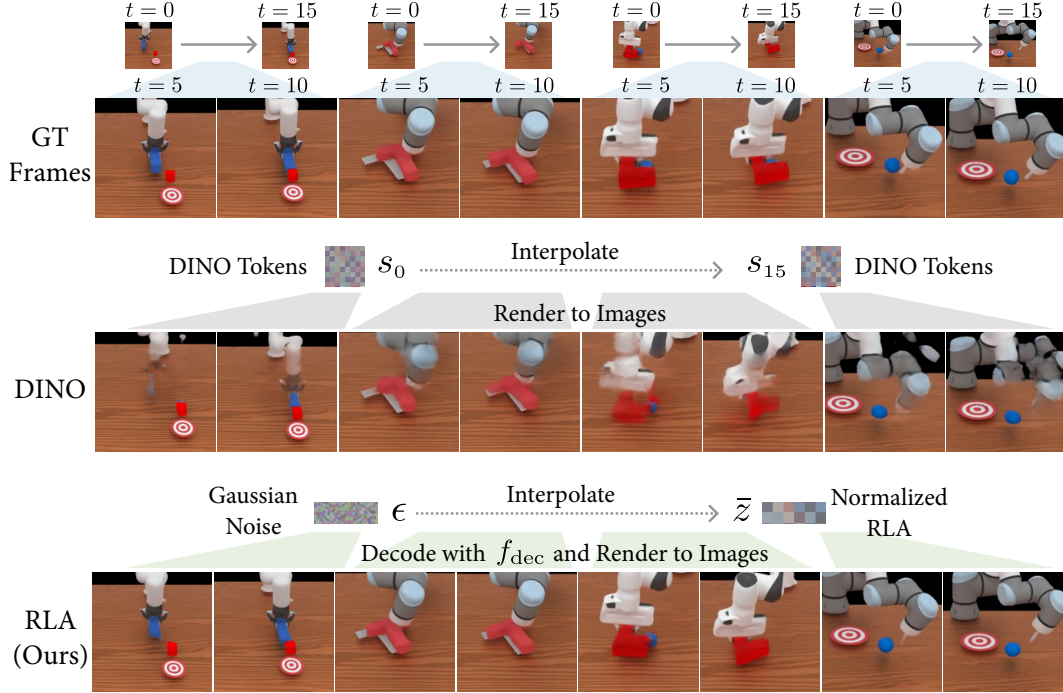


Figure A2: **RLA Temporal Topology.** We encode RLA  $z$  from  $(s_t, s_{t+h})$  and normalize it to  $\tilde{z} = (z - \mu)/\sigma$ , where  $\mu$  and  $\sigma$  are estimated from data. We then interpolate between Gaussian noise  $\epsilon$  and  $\tilde{z}$ , then denormalize and decode the result. For example,  $(\epsilon + \tilde{z})/2$  yields a reconstruction that approximates  $s_{t+h/2}$ . This indicates that the RLA latent space inherently captures temporal progression. In contrast, interpolating DINO tokens between  $s_t$  and  $s_{t+h}$  produces inferior results.

average pooling followed directly by an MLP for action prediction. The residual action head is a 3-layer MLP with hidden dimensions of 512 and 256. To stabilize exploration, the residual mean and standard deviation are bounded to 0.1 radians (joint angle) using tanh and softplus activations, respectively. For the Video Aligned Reward, the Poke Cube task uses the final goal frame  $s_T$ , while all other tasks compute the reward against the time-synchronized frame  $s_t$ . To augment initial state diversity, we initialize episodes from a random intermediate video frame with a 0.5 probability, controlled deterministically via the random seed. We apply a reward scale of 5 and rely exclusively on PPO gradients, without any terminal rewards and auxiliary behavior cloning losses.

RL optimization is conducted over 15 random seeds using a discount factor  $\gamma = 0.9$ , a GAE [72] parameter  $\lambda = 0.95$ , and a learning rate of  $1 \times 10^{-4}$ . We vector-parallelize 112 RLA-WM environments across multi-GPU setups. To ensure reproducibility, random seeds are uniquely mapped to the environment index rather than the GPU worker, so the same seed yields similar training results regardless of the number of GPUs. We rollout 300 steps inside the world model, each with 4 action-chunking steps of chunk size 8, and update the policy 8 times with a PPO batch size of 224. Every 200 updates we evaluate and save a checkpoint. Each training trial takes 3 hours on a 4-GPU A6000 machine or a 7-GPU A4500 Ada machine. We do not seed the Euler ODE steps for flow matching; although theoretically stochastic, we observe negligible effect on results (unlike environment resets or initial frame sampling). For final evaluation to select the best model for BC or RL, we run 1500 evaluation episodes with seeds 1-1500.

To bridge the *neural-to-sim* gap — the world model renders images from DINO tokens that differ slightly from simulator ray-traced images — we extract DINO tokens from each observed image, decode RGB images using our pre-trained UNet, and use the decoded image as observation. This largely removes the gap and is applied during both BC pre-training and evaluation. We acknowledge that this increases computational cost and may lower the performance ceiling, but we believe scaling the world model and policy will improve robustness to image styles in the future.



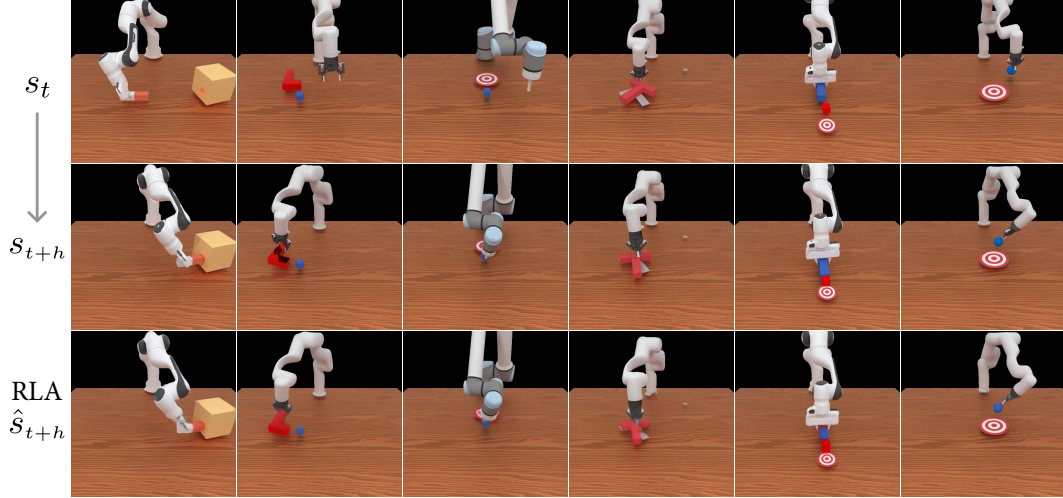


Figure A3: **RLA Generalization on Novel Tasks.** We apply the pre-trained RLA autoencoder to a previously unseen setup. For example, the original Panda robot is replaced with an XArm robot with a Robotiq gripper for the Pull Cube with Tool task. This interaction type was never observed during RLA autoencoder training. Yet, RLA maintains high reconstruction fidelity. Notably, this cross-embodiment generalization is achieved by training solely on the limited ManiSkill dataset, without relying on large-scale video pre-training of the RLA autoencoder.

### A.3 Limitations and Future Directions

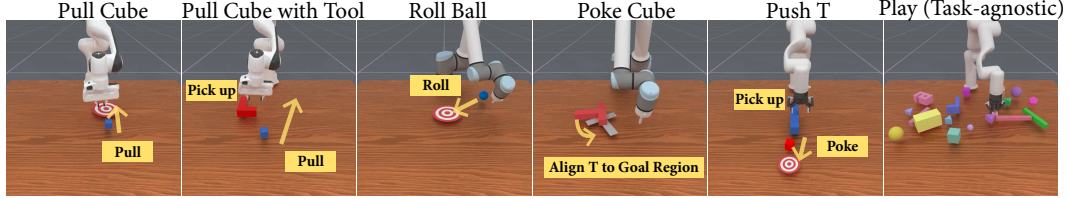
We summarize four key limitations and corresponding future directions.

1. **Background and random motion.** Our RLA is learned from residuals between pairs of DINO tokens  $(s_t, s_{t+h})$ . However, task-irrelevant background motion or workspace randomness (e.g., in humanoid robot or eye-in-hand camera) can also cause visual changes between  $s_t$  and  $s_{t+h}$ . Learning to encode those randomness-driven motions could waste representation capacity and degrade the RLA latent space. A promising fix is to move from 2D image learning to 3D, projecting DINO tokens into 3D [70] and learning view-independent 3D RLA.
2. **Memory and partial observability.** Our RLA-WM predicts  $s_{t+h}$  from  $s_t$  and  $a_{t:t+h}$ , yet changes may depend on  $s_{<t}$  due to occlusion (e.g., an object disappears and reappears). Because RLA  $z$  is learned from a single frame pair, it must memorize the object in the latent space rather than encoding true movement and occlusion events. Extending RLA to condition on multiple frames is a natural solution.
3. **Proprioceptive world model.** Our RLA-WM predicts only visual state evolution via RLA, but not future proprioceptive states. Proprioception input has been shown to be useful for policy learning. Extending the world model to predict both would broaden applicability.
4. **Scaling to larger datasets.** We deliberately evaluated on small-scale ManiSkill and IWS datasets to isolate method-driven gains from mere data scaling — many prior works scale first and leave it unclear whether improvements come from data volume or the method itself. Our clear, reproducible results on small data demonstrate the core properties of RLA and RLA-WM. Therefore, scaling to massive real-world datasets is a promising future step.

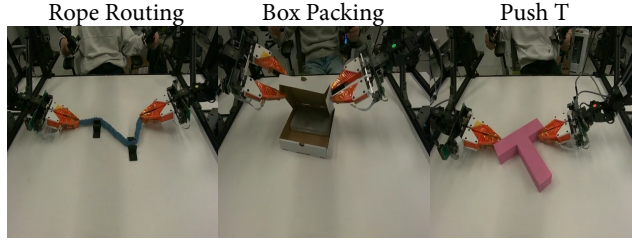
**Panda WMRL Results.** WMRL underperforms BC on the Panda robot for the Pull Cube and Pull Cube with Tool tasks. We train the dynamics component of RLA-WM per robot, yet only the Panda results show a consistent drop. We attribute this to a combination of factors: (1) the Panda’s kinematic structure, (2) the camera viewpoint, and (3) insufficient action diversity in the demonstration data.

The Panda arm has 8 degrees of freedom (DoF), whereas the XArm has 7 (the Robotiq gripper consists of 6 correlated joints that provide 1 effective DoF), and the UR10e has 5 (the sixth joint is

ineffective due to a cylindrical end-effector). Since our policy predicts future joint angles, higher-dimensional action spaces naturally require more data. However, all robots receive the same number of demonstrations. Moreover, the two tasks of Panda both involve pulling a cube toward the robot’s side of the table, producing action patterns with limited diversity. This pulling motion frequently causes occlusion from our front top-view camera, hindering the world model’s ability to capture the mapping from joint angles to visual changes accurately. Additionally, the Panda’s gripper fingers are small, making it difficult for the world model to capture fine visual details. We believe that adding multiple camera views, increasing task variety (and thus data diversity), and collecting more demonstrations overall would resolve these issues.



(a) ManiSkill Environment [49]



(b) IWS Dataset [10]

Figure A4: **Overview of Tasks and Datasets.** ManiSkill includes five tasks across three robots: Panda — Pull Cube (pull cube to target area) and Pull Cube with Tool (grasp L-shaped tool to hook cube within a distance to the robot); UR10 with cylinder end-effector Roll Ball (move ball to target region) and Push T (align T-shaped object with T-shaped goal area); XArm with Robotiq gripper Poke Cube (pick blue stick and poke cube to target). IWS (ALOHA robot) includes Rope Routing (route rope around marked anchors on table), Box Packing (open, close, or move the box), and Push T (continuous two-arm interaction with the T-object to create diverse movements).

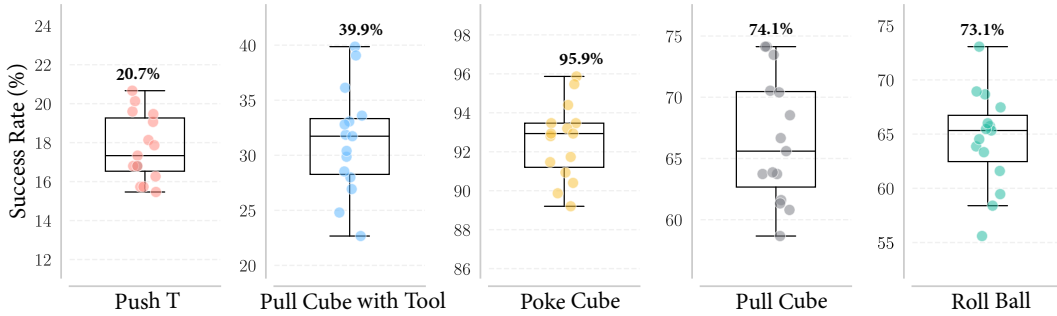


Figure A5: **WMRL Performance Distribution.** This plot complements Tab. 3 by showing the distribution of performance across 15 independent seeds (1-15). Each dot (•) represents the success rate of the best checkpoint for a given seed, evaluated on 1500 episodes per seed (seeds 1-1500).

Table A1: Detailed Per-Robot Evaluation of Future Frame Prediction on ManiSkill. Our RLA-WM achieves the best results across all robots and metrics.

| Model         | LPIPS ↓      |              |              | SSIM ↑       |              |              | DINO L1 ↓    |              |              |
|---------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|               | Panda        | XArm         | UR10         | Panda        | XArm         | UR10         | Panda        | XArm         | UR10         |
| DINO-WM       | 0.187        | 0.120        | 0.163        | 0.841        | 0.896        | 0.857        | 0.098        | 0.056        | 0.079        |
| RAE           | 0.310        | 0.313        | 0.349        | 0.719        | 0.730        | 0.701        | 0.139        | 0.166        | 0.124        |
| FM-WM         | 0.147        | 0.119        | 0.114        | 0.872        | 0.900        | 0.898        | 0.075        | 0.059        | 0.054        |
| Vid2World     | 0.219        | 0.186        | 0.192        | 0.685        | 0.720        | 0.710        | 0.094        | 0.081        | 0.077        |
| RLA-WM (Ours) | <b>0.091</b> | <b>0.061</b> | <b>0.061</b> | <b>0.912</b> | <b>0.941</b> | <b>0.941</b> | <b>0.035</b> | <b>0.028</b> | <b>0.027</b> |

Table A2: Detailed Per-Task Evaluation of Future Frame Prediction on IWS. Our RLA-WM achieves the best results on all tasks except rope routing, where it performs within a small margin of the best.

| Model         | LPIPS ↓      |              |              | SSIM ↑       |              |              | DINO L1 ↓    |              |              |
|---------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|               | Box          | PushT        | Rope         | Box          | PushT        | Rope         | Box          | PushT        | Rope         |
| DINO-WM       | 0.223        | 0.279        | <b>0.167</b> | 0.830        | 0.774        | <b>0.870</b> | 0.055        | 0.077        | <b>0.043</b> |
| RAE           | 0.596        | 0.523        | 0.530        | 0.549        | 0.669        | 0.656        | 0.167        | 0.153        | 0.157        |
| FM-WM         | 0.347        | 0.373        | 0.359        | 0.750        | 0.735        | 0.738        | 0.115        | 0.121        | 0.119        |
| Vid2World     | 0.384        | 0.398        | 0.382        | 0.716        | 0.702        | 0.714        | 0.134        | 0.142        | 0.142        |
| RLA-WM (Ours) | <b>0.206</b> | <b>0.204</b> | 0.177        | <b>0.839</b> | <b>0.845</b> | 0.857        | <b>0.055</b> | <b>0.055</b> | 0.048        |

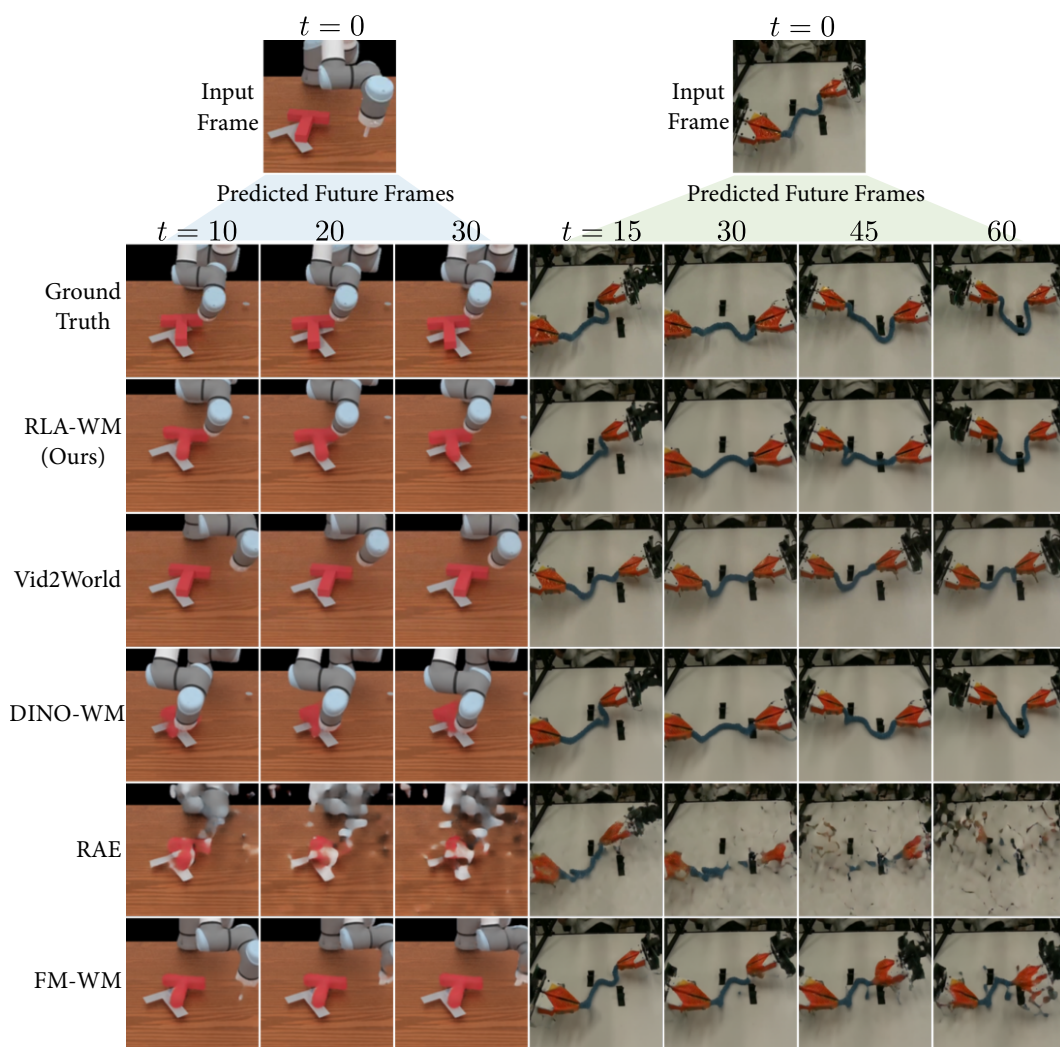


Figure A6: Additional Qualitative Comparison for RLA-WM.

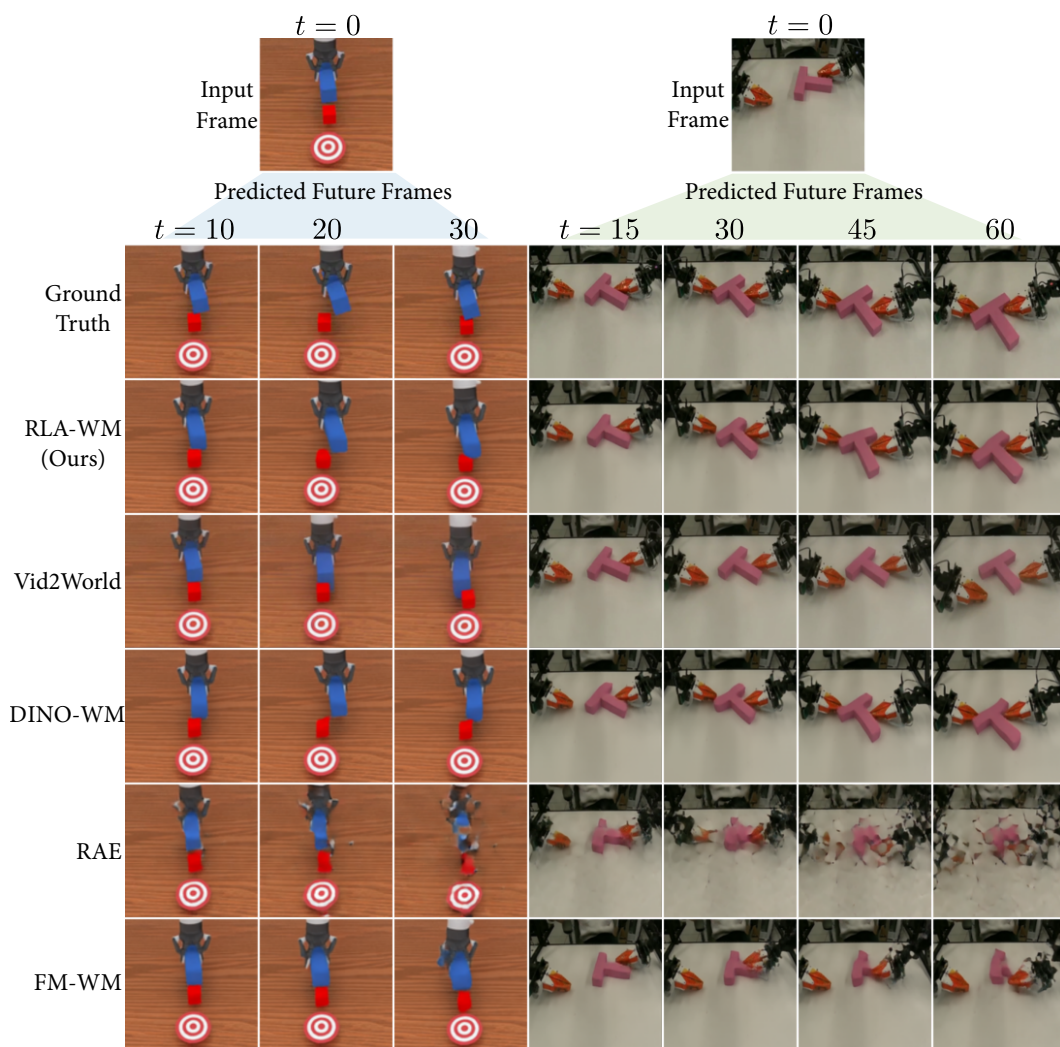


Figure A7: Additional Qualitative Comparison for RLA-WM.



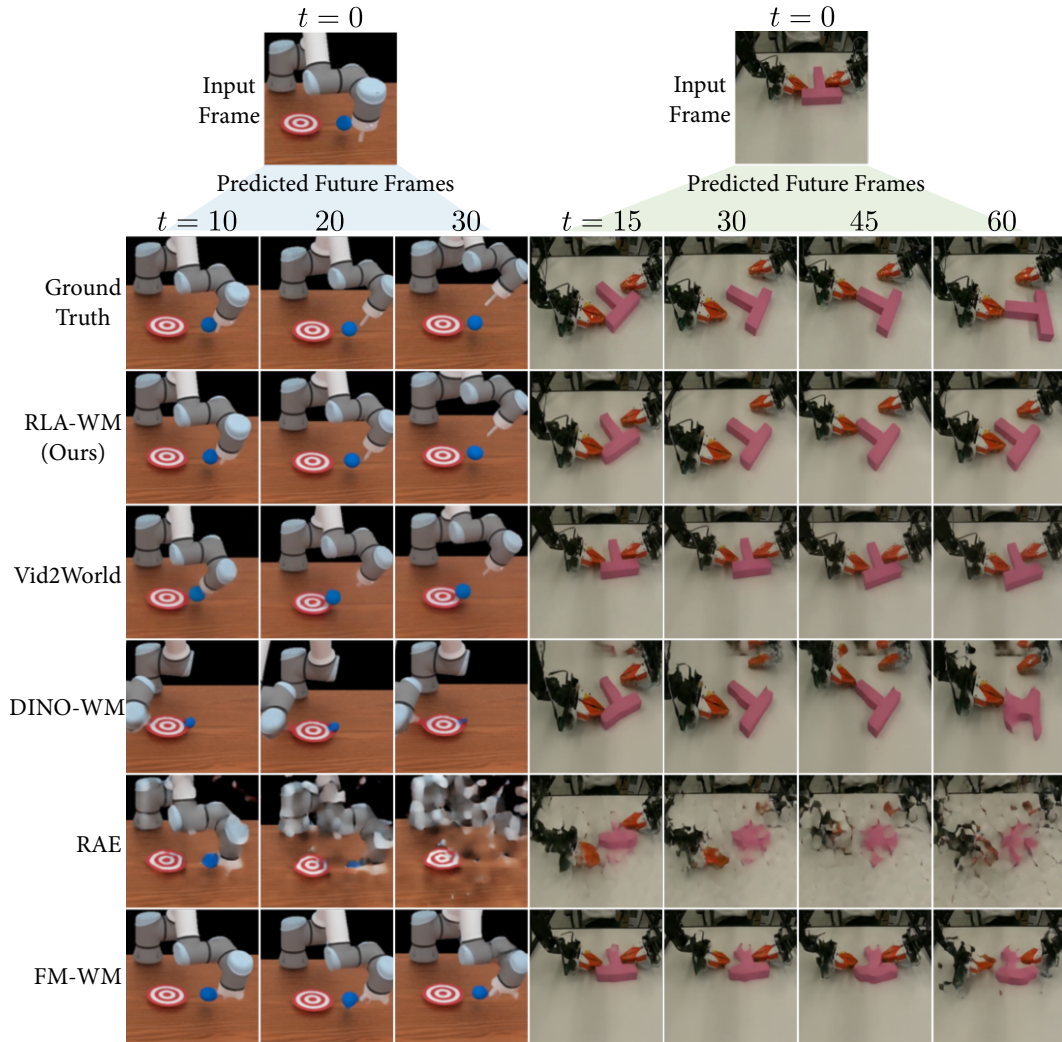


Figure A8: Additional Qualitative Comparison for RLA-WM.

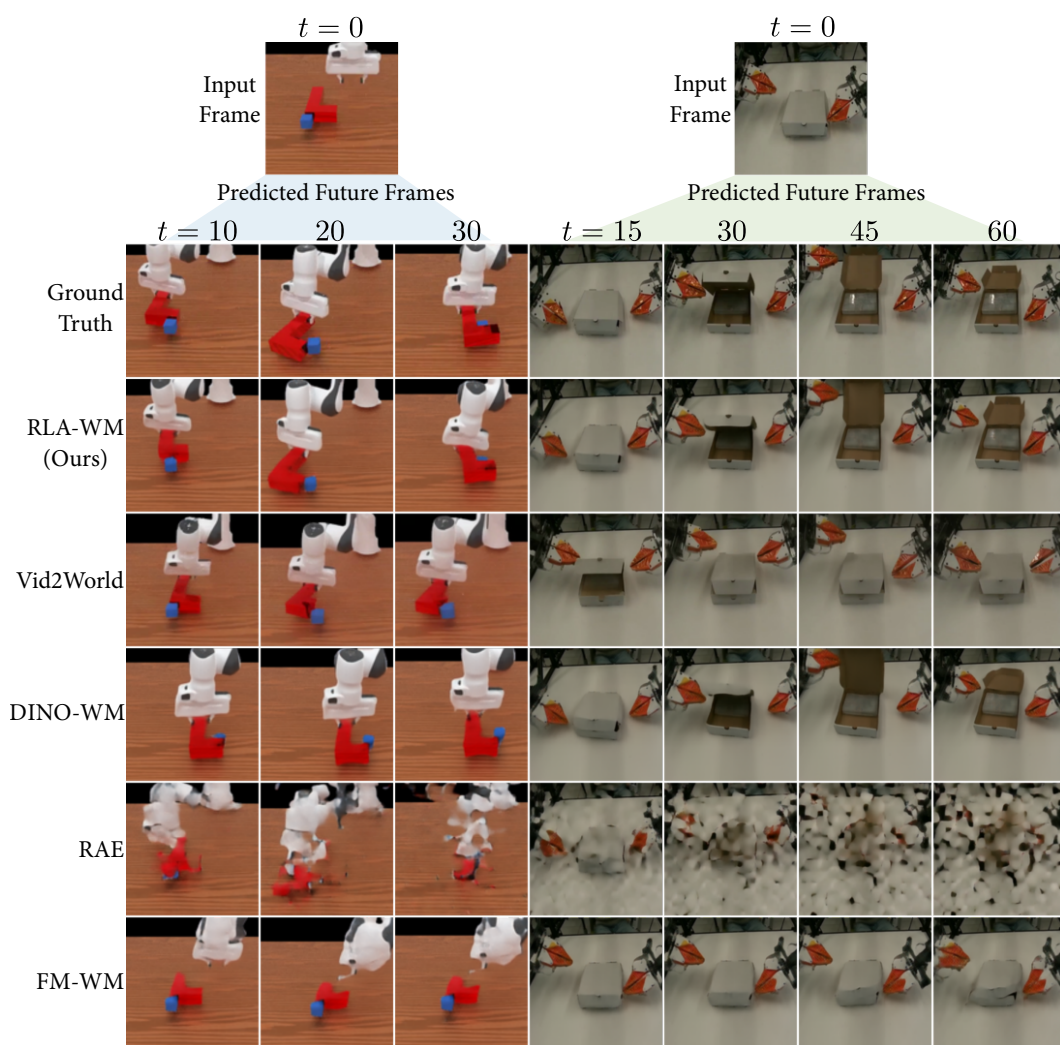


Figure A9: Additional Qualitative Comparison for RLA-WM.



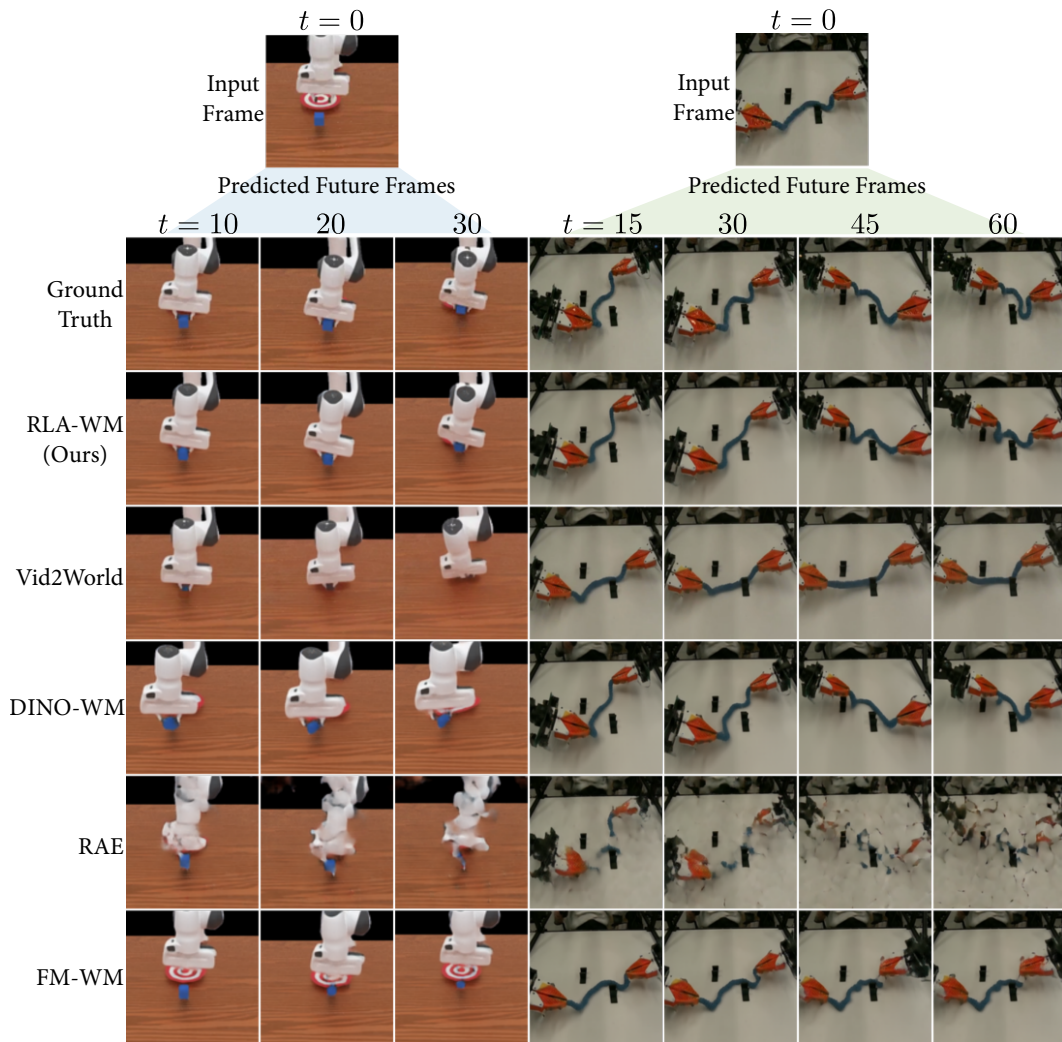


Figure A10: Additional Qualitative Comparison for RLA-WM.